Large Scale Social and Complex Networks: Design and Algorithms
ECE 232E — Summer 2026

Prof. Vwani Roychowdhury UCLA Department of Electrical and Computer Engineering

Course Project

Structured Memory Networks and Reasoning Inference Chain Retrieval

Based on PANINI: Continual Learning in Token Space via Structured Memory

100 points

Due Saturday, August 15, 2026 at 11:59 PM Pacific Time

1 Purpose and learning objectives

Standard retrieval-augmented generation stores document chunks and asks a language model to repeatedly reason over retrieved text. PANINI instead converts documents into GSWs: heterogeneous semantic networks containing entities, verb-phrase/event nodes, roles, states, and grounded question–answer (QA) edges. At inference time, RICR decomposes a multi-hop query and searches for compact chains of QA pairs through this structured memory.

This project connects the PANINI framework to core topics from ECE 232E: heterogeneous graph construction, network projection, connectivity, centrality, ranking, sparse and dense search, beam search, scaling, and controlled experimental evaluation. By the end of the project, you should be able to:

1. construct and characterize corpus-level networks from document-level GSWs;
2. compare TF–IDF, BM25, dense, hybrid, and structure-aware retrieval;
3. execute and validate a neural question-decomposition plan;
4. implement RICR with chain scoring and diversity-preserving pruning;
5. analyze generalization from 2WikiMultiHopQA to MuSiQue; and
6. report accuracy, retrieval quality, latency, evidence size, and failure modes reproducibly.

2 Reading, repository, and compute

Required reading. Read Sections 1–4 and Algorithm 1 of PANINI: Continual Learning in Token Space via Structured Memory. The GSW representation, dual indexing, decomposition notation, chain scoring, and RICR pruning rule in the paper are part of the project specification.

Student repository. The self-contained starter repository is:

`https://github.com/YigitTurali/panini-course-project`

The Colab notebook, artifact loaders, graph utilities, metrics, embeddings, indices, and model configuration are supplied. Students do **not** need access to the research codebase.

Models. Use the model identifiers in `models/model_config.json`. The required free-tier configuration uses:

- GSW-QA-Decomposer-Qwen3-4B for decomposition;
- supplied Qwen3-Embedding-8B corpus and fixed-query embeddings;
- Qwen3-Reranker-8B in 4-bit mode for reranking; and
- Qwen3-4B in 4-bit mode for evidence-grounded final answers.

The 8B decomposer is optional and receives no automatic accuracy advantage. Do not keep the decomposer, reranker, query encoder, and answer model resident on the GPU simultaneously. Save outputs to JSONL, unload the current model, clear GPU memory, and then begin the next stage.

Compute policy. The required path must run on a free Google Colab GPU. No paid API is required or permitted for the required results. Corpus GSWs, embeddings, and indices are supplied; regenerating them is outside the project scope.

3 Datasets and evaluation protocol

You will receive two independent 100-question packages with the same directory contract and stable IDs.

Dataset	Development	Held out	Total	Required stratification
2WikiMultiHopQA	80	20	100	25 bridge-comparison, 25 comparison, 25 compositional, and 25 inference questions; each category has 20 development and 5 held-out questions.
MuSiQue	80	20	100	50 two-hop, 30 three-hop, and 20 four-hop questions; development/held-out counts are respectively 40/10, 24/6, and 16/4.

Table 1: Required course subsets.

- The **development split** includes questions, answers, aliases, supporting evidence, and context-document IDs. It may be used for debugging and tables in the report.
- The **held-out split** includes questions and document IDs but withholds answers and supporting evidence. Do not manually search for, reconstruct, or import these labels.
- Select algorithms and default hyperparameters using only the 2Wiki development split. Freeze them before running MuSiQue. MuSiQue is the required cross-dataset transfer evaluation.
- Run the final frozen system once on all 200 questions. Report metrics locally for the 160 development questions; the teaching staff will score the 40 held-out predictions.

MuSiQue is distributed under CC BY 4.0; 2WikiMultiHopQA is distributed under Apache 2.0. Retain the dataset IDs and attribution files in all derived outputs.

4 Fixed definitions and default settings

4.1 Graph views and the reconciliation boundary

For each GSW, construct the **native directed multigraph**. Entity, space/time, and verb-phrase/event records are nodes. Each grounded QA answer is a directed edge from its verb-phrase node to its answer

node. The edge retains the stable QA ID, question, document ID, and direction. Do not merge records merely because their display strings match.

Next construct an **unreconciled entity projection**. Its nodes are the document-local entity occurrences identified by their stable UIDs. Two occurrences are adjacent when they answer QA records attached to the same local verb-phrase; repeated neighborhoods increment edge weight. Finally, construct one or more **analysis-only reconciled projections** by mapping occurrence UIDs to proposed cross-document entity IDs and aggregating the projected edges.

The supplied exact-surface baseline case-folds, removes punctuation, and collapses spaces. Students must implement and justify a more conservative reconciliation rule that also uses available type, role/state, and local neighborhood evidence. Neither method has complete gold identity labels, so evaluate it with a reproducible manual audit and a sensitivity comparison. All network results must name the precise graph and reconciliation rule used.

Why the reconciled graph is not used by PANINI. The paper reconciles entities within a document but does not precompute cross-document entity links. Its sparse entity index retains the originating document-local node, its dense index retrieves QA pairs directly, and RICR creates cross-document chains at read time by inserting a retrieved answer into the next sub-question. A global surface merge can incorrectly join homonyms, titles, years, occupations, and nationality values into artificial hubs; a conservative rule can instead miss aliases. These errors directly distort components and centralities and would contaminate retrieval if treated as facts. In this project reconciliation is therefore a network-analysis exercise only. Do not use reconciled edges for entity expansion, candidate generation, reranking, RICR, or answer prediction.

4.2 Retrieval and RICR defaults

Unless a question explicitly requests an ablation, use:

Random seed	232
Dense similarity	inner product of L2-normalized vectors
Reciprocal Rank Fusion	$\mathrm{RRF}(d) = \sum_r 1/(60 + r(d))$
Pre-rerank candidate pool	$M = 60$ unique QA records
Candidates returned per hop	$k = 15$
Beam width	$B = 5$
Chain score	geometric mean of hop scores
Generation	deterministic; sampling disabled

For a length-t chain $C = (e_1, \ldots, e_t)$ with positive hop scores $s_1, \ldots, s_t$, use

$$\mathrm{score}(C) = \left(\prod_{\ell=1}^{t} \max(s_\ell, \epsilon) \right)^{1/t} = \exp\left(\frac{1}{t} \sum_{\ell=1}^{t} \log \max(s_\ell, \epsilon) \right), \qquad \epsilon = 10^{-12}. \tag{1}$$

The log-domain form is recommended. At every pruning step, sort by decreasing chain score and use the stable QA-ID sequence as the deterministic tie-breaker. Retain at most one chain per normalized *current answer*.

4.3 Metrics

Use the provided normalization functions. Report:

Retrieval: Recall@k for $k \in \{1, 5, 10, 15\}$, reciprocal rank/MRR, supporting-document recall, and supporting-QA recall.

Chains: complete-chain recovery (all required supporting hops occur in one surviving chain), average surviving chains, and average unique current answers.

Answers: Exact Match (EM) and token F1, giving each question the maximum score over its accepted answer aliases.

Efficiency: wall-clock latency per stage, peak GPU memory when a model is used, number of reranked candidates, number of returned QA pairs, and answer-model input tokens.

Aggregate by macro-average over questions. Include the number of evaluated questions in every table. Never average already-rounded subgroup values.

5 Graded questions

5.1 Question 1: Understand the two data packages [4 points]

Imagine that the write-time stage of PANINI has just finished processing two collections of documents. You did not run that stage: you inherit its document-level GSWs, flattened entity and QA tables, embeddings, and indices. Before treating these files as one memory, you must establish what one row means and which identifiers remain valid when records from many documents are combined. A retrieval result that points to the wrong embedding row can look plausible while silently invalidating every later experiment.

Load both packages with `CoursePackage`. Use `entity_uid` and `qa_uid` as identifiers rather than list positions. Check that every embedding row has exactly one matching ID, IDs are unique, and held-out files contain inputs but no grading labels. This audit becomes the trusted boundary for all later questions.

(a) Produce a table containing, for each dataset and split, the number of questions, unique documents, GSW files, entities, QA records, and decomposition-validation examples.

(b) Show one question, entity, and QA record. Explain the stable ID and every field that your implementation consumes.

(c) Add an assertion that held-out records contain no answer-label fields: `answer`, `answer_aliases`, `supporting_facts`, or `evidences`.

(d) Run `pytest -q tests`. Record which tests pass and which RICR tests are skipped before you begin the implementation.

Required output. One dataset table, three short schema examples, the no-leakage assertion, and the initial test summary in the notebook.

Point allocation. Dataset table (1), schema explanation (1), automated leakage check (1), and starter-test run (1).

5.2 Question 2: Build the GSW network and reconcile entities [8 points]

The verified records from Question 1 are still hundreds of separate local workspaces. To study them as a network, first preserve exactly what was written inside each document, and only afterward ask whether two occurrences refer to the same real-world entity. Mixing those operations would make it impossible to tell whether a global edge came from a GSW or from your identity rule.

Construct the native directed multigraph by namespacing every node with its document and GSW filename. Add entity, space/time, and verb-phrase nodes. For each grounded QA answer, add a verb-phrase-to-answer edge keyed by QA ID and retain question text and provenance. Then project each local verb neighborhood onto its answer entities without merging across documents. Reconciliation must

remain a separate occurrence-UID-to-global-ID mapping so that the same unreconciled graph can be evaluated under several identity hypotheses.

(a) Report native-graph node and edge counts by node/edge type and verify them against direct counts from the JSON files.

(b) Construct the weighted unreconciled entity projection. Verify that occurrence UIDs from different documents remain distinct, even when their names match.

(c) Run the exact-surface baseline, then implement a conservative reconciliation rule using surface, type/role compatibility, and local neighborhood evidence. State the decision rule precisely.

(d) Manually audit a fixed-seed sample of at least 30 proposed cross-document merges. Label each correct, incorrect, or uncertain; report estimated precision with uncertain cases shown separately. Include at least two false-merge and two missed-alias examples.

(e) Add unit tests for edge direction, provenance, projection weight, cross-document non-merge, intended merge, and homonym non-merge.

Required output. Construction and reconciliation code, the audit table, graph counts under all three identity conditions, and passing tests.

Point allocation. Correct native graph (2), correct unreconciled projection (2), reconciliation method and audit (3), and tests (1).

5.3 Question 3: Analyze the structured-memory network [8 points]

Question 2 produces several candidate "global memories," but network statistics can make a bad reconciliation look convincing. For example, merging every occurrence of a nationality, year, occupation, or ambiguous title can manufacture a giant component and high-centrality hubs that were never asserted by any document. Your task is therefore not merely to compute statistics; it is to decide which observed structure belongs to the local GSWs and which was created by the reconciliation rule.

Perform the following analysis separately for 2Wiki and MuSiQue. Compute every requested property on the unreconciled projection, exact-surface projection, and conservative projection; include native-graph measurements as a reference.

(a) Compute weakly connected components of the native graph and connected components of each entity projection. Report the number of components, giant-component size and fraction, isolated nodes, and component-size summary (minimum, median, mean, and maximum). Quantify how much each reconciliation rule changes these values.

(b) Plot degree probability mass and complementary cumulative degree distributions on log–log axes. State whether the heavy tail, if any, is consistent across datasets and reconciliation rules; do not claim a power law without a fitted test.

(c) For each reconciled projection, report the ten highest nodes under weighted degree, PageRank, and betweenness centrality. Audit whether hubs represent entities, common attribute values, or reconciliation errors. Exact betweenness or a documented fixed-seed approximation is acceptable.

(d) Report average clustering and degree assortativity, then visualize one complete gold multi-hop path from a development question with node types and edge/QA labels. Conclude with a specific explanation of why the reconciled projection must not be used by the PANINI retrieval pipeline.

Required output. At least four plots, two sensitivity tables, and a 200–300 word cross-dataset interpretation that separates local GSW structure from reconciliation artifacts.

Point allocation. Connectivity sensitivity (2), degree analysis (2), centrality audit (2), and clustering/assortativity/path interpretation (2).

5.4 Question 4: Question decomposition and dependency graphs [10 points]

The network audit does not yet answer a user's multi-hop question. RICR first turns that question into an executable plan. For example, "Which director died later?" may create two independent branches that retrieve each director and death date, followed by a deterministic comparison. The decomposer emits text, but the retrieval system needs a graph whose dependencies and operation types are explicit.

Run the supplied 4B decomposer on all 200 questions using the supplied prompt, sampling disabled, and the model-config maximum output length. Cache the raw output before parsing so a parser failure never requires another model run. Parse every numbered sub-question into a template node. A placeholder such as `<ENTITY_Q1>` creates an edge from Q1 to the dependent template. Reject duplicate IDs, missing or future references, cycles, and unresolved placeholders. Mark retrieval nodes separately from deterministic operations such as comparison or intersection, then topologically sort the result.

(a) Validate references: every placeholder must point to an earlier sub-question, the graph must be acyclic, and every retrieval node must contain a nonempty natural-language question.

(b) Identify independent linear sequences that may run in parallel and record the deterministic operations needed to combine their answers.

(c) On the public reviewed decomposition subset, report valid-plan rate, sub-question-count exact match, dependency-edge precision/recall/F1, and retrieval-vs-reasoning node accuracy.

(d) Analyze two errors from each dataset. Distinguish malformed output, wrong dependency, missing hop, extra hop, and semantically incorrect atomic question.

Required output. `decompositions.jsonl`, parser tests, a metric table, and four concise error analyses.

Point allocation. Inference and caching (2), parser/validation (3), parallel-sequence handling (2), metrics (2), and error analysis (1).

5.5 Question 5: Sparse retrieval baselines [8 points]

The dependency graph now tells the system what atomic question to ask, but not where to find its answer. Begin with sparse methods because their behavior is transparent: when a result fails, you can inspect the matched terms and decide whether the problem is wording, entity description, or graph expansion. These methods also establish whether later neural components earn their additional cost.

Give every backend the same return format, `(qa_uid, score, rank, source)`, and build it first on a tiny hand-written corpus with one obvious answer. Using only the supplied metadata, implement and evaluate:

(a) TF–IDF retrieval over concatenated QA question and answer text;

(b) BM25 retrieval over the same QA text; and

(c) BM25 entity retrieval over surface name, aliases, roles, and states, followed by expansion to QA records attached to each retrieved entity.

Use identical text normalization within a method across datasets. State all tokenization and BM25 parameters. Evaluate the gold atomic sub-questions from the development splits at $k \in \{1, 5, 10, 15\}$. Report Recall@k and MRR overall, by 2Wiki question type, and by MuSiQue hop count.

Required output. One implementation per method, a complete retrieval table, and a short explanation of where entity expansion helps or hurts.

Point allocation. Correct TF–IDF/BM25 QA search (3), correct entity expansion (2), evaluation (2), and interpretation (1).

5.6 Question 6: Dense, hybrid, and paper-style dual retrieval [12 points]

Sparse retrieval exposes a central difficulty of multi-hop search: an atomic question may paraphrase the stored QA text, while an entity mention may be the best doorway into a relevant local workspace. PANINI addresses these two failure modes with two independent routes—dense QA retrieval and sparse entity retrieval followed by local expansion—and joins them before reranking. Notice that this expansion uses document-local GSW adjacency from Question 2, never the analysis-only reconciled graph.

Load the supplied normalized QA embeddings and FAISS index; do not regenerate corpus embeddings. Use the supplied fixed-query embedding when present. If answer substitution creates a new query, encode it with the configured query encoder and cache it by exact normalized query string.

(a) Implement dense QA retrieval and verify that a manual inner-product calculation agrees with FAISS for at least five queries.

(b) Implement RRF over TF–IDF, BM25-QA, and dense rankings with constant 60. Deduplicate by stable QA ID.

(c) Implement paper-style dual retrieval: union QA records obtained from BM25 entity expansion and dense QA search, deduplicate to a pre-rerank pool of at most $M = 60$, rerank against the instantiated atomic question, and return the top $k = 15$.

(d) Compare all methods using the Question 5 protocol. Add mean and 95th-percentile retrieval latency and the average candidate count.

Required output. Search code, FAISS consistency test, performance/latency tables, and the saved top-15 retrieval trace for every development atomic question.

Point allocation. Dense retrieval (3), RRF (2), dual candidate construction (3), reranking/deduplication (2), and evaluation (2).

5.7 Question 7: Does reranking always help? [5 points]

Question 6 deliberately constructs a broad candidate pool: its goal is recall, so high-ranked items from different routes may be redundant or only superficially related. The reranker is supposed to repair that ordering using the instantiated atomic question. It can also overrule a strong retrieval signal and move the first supporting QA downward. Treat reranking as a hypothesis to test, not an automatically beneficial step.

Freeze each query's $M = 60$ candidate pool so every method sees exactly the same QA IDs. Compare three scoring rules:

(i) reranker probability only;

(ii) reciprocal retrieval rank only; and

(iii) 0.5 reranker probability + 0.5 reciprocal retrieval rank.

Use rank reciprocal $1/r$ after assigning each candidate its best pre-rerank rank. Report Recall@15 and MRR for each dataset. Find one query for which reranking improves the first relevant rank and one for which it worsens it. For each, show the top five candidates before and after scoring and explain the lexical/entity mismatch.

Required output. One comparison table and two annotated retrieval traces.

Point allocation. Controlled comparison (2), traces (2), and evidence-based explanation (1).

5.8 Question 8: Implement Reasoning Inference Chain Retrieval [15 points]

At this point you have an executable decomposition and a retriever for one atomic question. The remaining problem is sequential: the answer selected at one hop changes the text of the next query. Greedily keeping only the highest-scoring first answer can destroy the correct chain, while retaining every candidate causes exponential growth. RICR resolves this tension with a small beam whose chains carry their own instantiated questions, evidence, and scores.

Complete `prune_unique_answers` and `run_linear_ricr` in `panini_course/ricr.py`. Represent a chain as an immutable ordered tuple of evidence records, current answer, instantiated questions, and hop scores. Implement the search first with a dictionary-backed toy retriever whose outputs you control, following this structure:

```
beams = retrieve(first_instantiated_question, k)
beams = prune_unique_answers(beams, B)
for template in remaining_templates:
    expansions = []
    for chain in beams:
        query = substitute(template, chain.current_answer)
        for candidate in retrieve(query, k):
            expansions.append(new_chain(chain, query, candidate))
    beams = prune_unique_answers(expansions, B)
```

For each linear sequence:

(1) retrieve k candidates for the first instantiated sub-question;

(2) retain the top B chains with distinct normalized current answers;

(3) substitute each current answer into the next dependent sub-question;

(4) expand all surviving chains by k candidates;

(5) update the geometric-mean chain score in the log domain;

(6) globally prune the kB expansions to B distinct current answers;

(7) repeat until the sequence ends; and

(8) combine parallel sequences by deduplicating evidence using stable QA ID while preserving the highest-scoring provenance.

A candidate from an earlier chain may not be mutated in place. Pruning is global across all kB expansions, not performed separately per parent. Sort by decreasing geometric-mean score and then stable QA-ID sequence, retaining only the first chain for each normalized current answer. Ties must use the deterministic rule from Section 4.2. Return both the deduplicated evidence and complete trace: instantiated questions, candidates, scores, pruned chains, and selected QA IDs at every hop. Only after toy cases pass should you connect the real retriever from Question 7.

Required output. Working implementation, all supplied RICR tests passing, at least three additional tests of your own (including parallel sequences), plus one human-readable trace from each dataset.

Point allocation. First-hop/pruning behavior (3), multi-hop expansion/substitution (4), chain scoring (2), parallel evidence merge (2), deterministic trace format (1), and tests/edge cases (3).

5.9 Question 9: RICR ablation study [10 points]

A working search procedure does not show that its design choices are necessary. A larger beam can preserve a correct early hypothesis but increases retrieval calls; unique-answer pruning buys diversity but may discard useful duplicate evidence; geometric means prevent long chains from winning merely because they have more terms. Use controlled ablations to determine which mechanisms produce useful accuracy rather than accidental complexity.

Select a fixed, stratified 20-question subset from each development dataset and publish the IDs before running experiments. The default is $B = 5$, $k = 15$, unique-answer pruning enabled, geometric-mean scoring, and the best frozen Question 7 retrieval score.

(a) Beam width: $B \in \{1, 3, 5\}$.

(b) Candidates per hop: $k \in \{5, 15\}$.

(c) Diversity: unique-answer pruning on versus off.

(d) Chain scoring: geometric mean versus last-hop score.

(e) Retrieval: BM25-QA, dense QA, RRF hybrid, and paper-style dual retrieval.

Report supporting-QA recall, complete-chain recovery, answer EM/F1, mean latency, and mean evidence count. Plot accuracy versus latency and accuracy versus evidence size. Change only the named factor in each comparison.

Required output. One ablation table, two trade-off plots, and a 200–300 word recommendation of the configuration you would deploy.

Point allocation. Experimental control (2), beam/candidate ablations (2), diversity/scoring ablations (2), retrieval ablation (2), and trade-off analysis (2).

5.10 Question 10: End-to-end 2Wiki evaluation [7 points]

The component experiments now become one complete system in its development domain. Select the final configuration using only labeled 2Wiki development questions, record that choice, and freeze it before producing any held-out prediction. This separation matters: otherwise the final score measures continued tuning rather than generalization.

Run the frozen system on all 100 2Wiki questions. The answer model may see only the deduplicated QA evidence returned by RICR, never source documents, held-out labels, or reconciled-graph neighbors. Its prompt must request `N/A` when the evidence is insufficient. Save the trace before generating the answer so retrieval and generation failures can be distinguished.

On the 80 labeled questions, report decomposition validity, supporting-QA recall, complete-chain recovery, EM, F1, average/95th-percentile latency, answer-context tokens, and evidence count. Break out EM/F1 by the four 2Wiki question types. Include one successful and one failed complete trace.

Required output. `results/2wiki_dev.jsonl`, `predictions/2wiki_heldout.jsonl`, one main table, one type table, and two traces.

Point allocation. Correct frozen run/output files (2), complete metrics (2), type-specific analysis (1), trace quality (1), and grounded answer prompting (1).

5.11 Question 11: MuSiQue transfer and scaling evaluation [8 points]

The final test is whether the system learned a reusable procedure or a set of 2Wiki-specific choices. MuSiQue changes both the source distribution and chain length, including three- and four-hop questions. Treat it as a deployment to a new collection: package paths may change, but the algorithm and parameters may not.

Without changing retrieval weights, M, k, B, prompts, normalization, or score rules, replace the 2Wiki package with the MuSiQue package and run all 100 questions. On the 80 labeled questions:

(a) report the same metrics as Question 10, separated into 2-, 3-, and 4-hop groups;

(b) plot complete-chain recovery and F1 against hop count;

(c) compare 2Wiki and MuSiQue under the same frozen configuration; and

(d) identify whether the largest transfer loss comes from decomposition, first-hop retrieval, later-hop substitution/retrieval, or answer generation, using counts from saved traces rather than intuition.

Required output. Submit:

- `results/musique_dev.jsonl` and `predictions/musique_heldout.jsonl`;
- a hop-count table and two scaling plots; and
- a quantitative transfer-error analysis.

Point allocation. Frozen transfer run (2), hop-stratified metrics (2), scaling plots (1), cross-dataset comparison (1), and traced error attribution (2).

5.12 Question 12: Reproducibility and final submission [5 points]

Your final artifact should allow another student to reproduce a result from a fresh Colab runtime and to inspect where a wrong answer arose. Treat this as a handoff of the system rather than a notebook snapshot: paths must be relative, expensive stages restartable, configurations frozen in files, and predictions separate from gold labels.

Submit one archive or course-repository commit containing:

(a) `report.pdf`, organized by Questions 1–12 and limited to 12 pages excluding references and a one-page appendix;

(b) the completed Colab notebook and any Python modules/tests it imports;

(c) all four development/held-out JSONL files from Questions 10–11;

(d) `environment.txt` with Python, CUDA, package, and model revisions; and

(e) `RUNME.md` with exact commands, seeds, expected runtime, restart points, and the final Git commit hash.

The notebook must run from a fresh Colab runtime using relative paths after setting one `PACKAGE_ROOT`. Do not submit downloaded model weights, provided embeddings, FAISS indices, or dataset copies.

Point allocation. Report organization/completeness (2), executable code/tests (1), required result schemas (1), and environment/RUNME reproducibility (1).

6 Required result schema

Each prediction file must contain one JSON object per line and preserve the input order. Extra diagnostic fields are allowed, but the following fields are required:

```
{
  "dataset": "2wiki" | "musique",
  "split": "development" | "held_out",
  "question_id": "...",
  "question": "...",
  "predicted_decomposition": [...],
  "decomposition_valid": true,
  "retrieval_backend": "dual_hybrid",
  "beam_width": 5,
  "candidates_per_hop": 15,
  "chains": [
    {
      "qa_ids": ["...", "..."],
      "answers": ["...", "..."],
      "hop_scores": [0.91, 0.87],
      "chain_score": 0.8898
    }
  ],
  "evidence_qa_ids": ["...", "..."],
  "predicted_answer": "...",
  "latency_ms": {
    "decomposition": 0.0,
    "retrieval_ricr": 0.0,
    "answer": 0.0
  },
  "answer_context_tokens": 0
}
```

Development files may additionally contain gold labels and computed metrics. Held-out files must not contain gold fields.

7 Grading summary

Component	**Points**
Q1. Understand the two data packages	4
Q2. GSW network and entity reconciliation	8
Q3. Structured-memory network analysis	8
Q4. Question decomposition and dependency graphs	10
Q5. Sparse retrieval baselines	8
Q6. Dense, hybrid, and dual retrieval	12
Q7. Reranking analysis	5
Q8. RICR implementation	15
Q9. RICR ablations	10
Q10. 2Wiki end-to-end evaluation	7
Q11. MuSiQue transfer and scaling	8
Q12. Reproducibility and submission	5
Total	**100**

8 General grading and conduct

- A table or plot without axis labels, units, evaluated-question count, or a caption receives at most half credit for that item.
- Results that cannot be reproduced from the submitted code receive no implementation credit, even if the reported values are plausible.
- Clearly label any deviation from the required settings. An additional experiment does not replace a required one.
- You may use standard libraries for graph storage, TF–IDF, BM25, FAISS, plotting, and model loading. You may not import an implementation of RICR or copy an answer key.
- Follow the course collaboration and academic-integrity policy. Cite all external code, prompts, and written sources.
- The course late-submission policy applies. If a released artifact is corrupted or a Colab dependency becomes unavailable, post a minimal reproducer to the course forum before the deadline.

The goal is not merely to obtain a final answer. Your system should make the network path, evidence, score, and failure mode inspectable.